

Журнал звонков Panasonic KX-TDE200 — документация

Система собирает данные о звонках с АТС Panasonic KX-TDE200 по протоколу SMDR (TCP), сохраняет их в базу данных и отображает в веб-интерфейсе с фильтрами, статистикой и справочником сотрудников.

Заменяет собой сломанный eSMDR (RS-232), используя вместо него SMDR-вывод по LAN — новый метод в настройках самой АТС.

1. Архитектура

Система состоит из двух независимых частей, работающих постоянно в фоне:





Браузер (веб)

- `collector.py` — держит постоянное TCP-соединение к АТС, авторизуется (логин `SMDR` , пароль `CCSMDRP`), разбирает построчный поток SMDR и пишет каждый звонок в `smdr.db` . При обрыве связи переподключается каждые 5 секунд. Ничего не показывает пользователю — работает тихо в фоне.
- `server.js` — веб-интерфейс на Node.js/Express. Читает ту же базу, показывает журнал с фильтрами, статистику, справочник добавочных. Закрывает логином/паролем (один общий на всех).
- `smdr.db` — общая база SQLite, лежит в `C:\SMDR\smdr.db` , используется обоими процессами одновременно.

Оба процесса подняты как **службы Windows** **через NSSM** — стартуют сами при загрузке компьютера, не требуют открытых окон терминала.

2. Дерево файлов на сервере

```
C:\SMDR\
|
├─ collector.py          ← сборщик
звонков (Python)
├─ collector.log         ← лог
сборщика (RAW/CALL/HE РАСПОЗНАНО/обрывы
связи)
├─ smdr.db              ← база
данных (общая для collector.py и веба)
├─ venv\                ←
виртуальное окружение Python
|
├─ web-node\            ← веб-
интерфейс (Node.js)
    ├─ server.js         ←
основной сервер (роуты, API, авторизация)
    ├─ db.js             ←
подключение к smdr.db + функция lower_ru
    ├─ package.json
    ├─ .env              ← логин/
пароль веба, секрет сессии, порт
    ├─ node_modules\
    ├─ views\
    │   ├─ login.ejs
    │   └─ dashboard.ejs ←
журнал звонков (главная страница)
    │   └─ stats.ejs     ←
статистика, графики, сводная таблица
    │   └─ directory.ejs ←
справочник добавочный → ФИО
    └─ _rows.ejs        ←
```

```
partial со строками таблицы журнала
  |   └─ _theme_init.ejs      ←
partial применения тёмной/светлой темы
  └─ public\
    └─ style.css              ← все
стили (светлая/тёмная тема)
    └─ app.js                ←
переключение темы
```

3. Настройки АТС (не менять без необходимости)

В Maintenance Console: **Maintenance** → **Main** → **System** → **SMDR Options**

Параметр	Значение
Output method	LAN
Destination IP	IP хоста Windows, где крутится <code>collector.py</code>
Port	23000
Outgoing/Incoming Call Print	Print

АТС сама открывает TCP-соединение к коллектору, шлёт `\r\n`-разделённые

строки, требует логин (SMDR) и пароль (CCSMDRP) сразу после подключения.

Важно: АТС разрешает только одно активное подключение к SMDR-порту одновременно. Если запущено два коллектора (или тестовый скрипт + основной) — АТС будет рвать соединения. Держите запущенным только collector.py как службу.

4. Формат данных SMDR (как он разбирается)

Примеры сырых строк, которые шлёт АТС:

```
06/08/26 08:54 3011 06
<D>358623<I>984742420323 0'03 00:01'21
← входящий
06/08/26 08:54 2042 30 89610358265
00:00'34 ← исходящий
06/08/26 08:55 1016 EXT4041
← внутренний
06/08/26 09:06 2091 13
<D>358601<I>989997501396 0'04 00:00'00 NA
← пропущенный
```

Разбор (collector.py , функция parse_line):

Поле	Что значит
<code>ext</code>	Добавочный, на котором произошло событие
<code>co</code>	Номер линии/транка
<code><D>...</code> <code><I>...</code>	Входящий: <code>D</code> — DID-номер (куда звонили), <code>I</code> — номер звонящего
<code>EXTxxxx</code>	Внутренний звонок — на какой добавочный переведено
<code>ring</code>	Время до ответа (например <code>0'04</code> = 4 сек)
<code>duration</code>	Длительность разговора чч:мм'сс
<code>NA (status)</code>	No Answer — звонок не был отвечен (используется для бейджа "Пропущенный")

Строки, которые не подошли ни под один шаблон, не теряются — сохраняются в БД с `direction = 'unknown'`, и попадают в `collector.log` с пометкой `НЕ РАСПОЗНАНО:`. Если увидите такие в логе — пришлите разработчику конкретную сырую строку для донастройки парсера.

5. База данных (таблица `calls`)

Поле	Описание
<code>call_dt</code>	Дата+время звонка в формате <code>YYYY-MM-DD HH:MM:00</code> (секунд АТС не передаёт)
<code>ext</code>	Добавочный
<code>co</code>	Линия
<code>direction</code>	<code>outgoing</code> / <code>incoming</code> / <code>internal</code> / <code>unknown</code>
<code>did</code> , <code>caller</code>	Для входящих: DID-номер и номер звонящего
<code>internal_to</code>	Для внутренних: на какой добавочный переведено
<code>number</code>	Для исходящих: набранный номер
<code>ring</code> , <code>duration</code>	Ожидание ответа / длительность разговора
<code>status</code>	<code>NA</code> для пропущенных, иначе пусто

Поле	Описание
<code>raw_line</code>	Исходная строка от АТС (для отладки)

Таблица `employees` — простой справочник `ext` → `fio`, заполняется вручную через страницу "Справочник" в вебе.

6. Управление службами (NSSM)

Обе службы — `SMDR_Collector` (Python) и `SMDR_Web` (Node.js) — запускаются автоматически при старте Windows.

```
nssm start    SMDR_Collector    —
запустить
nssm stop     SMDR_Collector    —
остановить
nssm restart  SMDR_Collector    —
перезапустить

nssm start    SMDR_Web
nssm stop     SMDR_Web
nssm restart  SMDR_Web
```

Или через **Панель управления** → **Администрирование** → **Службы** (`services.msc`) — там же можно посмотреть,

запущены ли они,
и настроить через GUI.

Когда нужен перезапуск службы

Что поменяли	Нужен ли рестарт
<code>collector.py</code>	Да — <code>nssm restart SMDR_Collector</code>
<code>server.js</code> , <code>db.js</code>	Да — <code>nssm restart SMDR_Web</code>
<code>.ejs</code> (шаблоны страниц), <code>.css</code> , <code>.js</code> во <code>public\</code>	Нет — просто обновить страницу в браузере
<code>package.json</code> (добавили новую библиотеку)	Да, и перед этим <code>npm install</code>

7. Типовые проблемы и решения

Веб не открывается (`localhost:5001` не отвечает)

→ Проверьте `nssm status SMDR_Web` в CMD, посмотрите на вкладке "I/O" в настройках службы, куда пишутся

stdout/stderr — там будет
текст ошибки Node.

В `collector.log` постоянно "Обрыв связи: АТС закрывает соединение"

→ Скорее всего где-то ещё открыто второе
подключение к порту 23000
(например, забытое тестовое окно PowerShell).
АТС не пускает второе
подключение одновременно. Закройте лишние
подключения, оставьте
только службу.

Пропущенные звонки не помечаются бейджем

→ Проверяйте по полю `status = 'NA'`, а не по
нулевой длительности —
нулевая длительность бывает и у отвеченных
звонков (разговор доли секунды).

Веб не находит `smdr.db`

→ `db.js` ищет базу по относительному пути
`../smdr.db` от папки
`web-node`. Структура папок должна быть строго
`C:\SMDR\smdr.db` +
`C:\SMDR\web-node\`, не переносите `web-node` в
другое место без
правки пути в `db.js`.

После `npm install` ошибка компиляции `better- sqlite3` (нужен Visual Studio)

→ Значит версия пакета в `package.json` не имеет

готового бинарника
под установленную версию Node.js. Проверьте
`node --version` и
актуальную поддержку `better-sqlite3` на npm —
возможно, нужно
поднять версию пакета в `package.json`.

Статистика (графики) не грузится

→ `Chart.js` подключается с локального сервера
(`/vendor/chart.umd.js` ,
раздаётся из `node_modules/chart.js/dist`), а не с
внешнего CDN —
специально, чтобы не зависеть от блокировок в
сети. Если всё равно
не грузится — проверьте, что `chart.js` есть в
`node_modules`
(`npm install` после обновления `package.json`).

8. Логины/пароли

- **SMDR (АТС → коллектор):** логин `SMDR` , пароль `CCSMDRP` — заданы
протоколом Panasonic, не наши, менять не
нужно.
- **Веб-интерфейс:** логин/пароль в `C:\SMDR\web-node\.env`
(`WEB_LOGIN` , `WEB_PASSWORD`). Один общий на
всех пользователей.

Смена пароля — правка `.env` + `nssm restart SMDR_Web` .

9. Контакты / история изменений

Система собрана поэтапно в переписке с Claude (Anthropic) в августе 2026:

от первого TCP-подключения к SMDR-порту ATC — до готового веб-приложения с авторизацией, реалтайм-обновлением, статистикой и тёмной темой.

При доработках — сохраняйте эту логику:

`collector.py` только собирает и парсит сырые данные в БД, `web-node` только читает БД и отображает, они не должны напрямую общаться друг с другом, только через `smdr.db` .